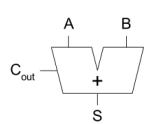


What do the two adder outputs mean?

Week 6/Chapter 4 Arithmetic for Computers.pdf, p.3. Isolate the two adder symbols; remove the displayed equations and completed truth tables. For the fourth turn, a small *unanswered* two-bit ripple-chain drawing would make the connection visible.

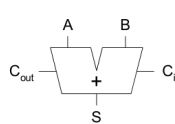
Half Adder



A	B	C _{out}	S
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	0

$S = A \oplus B$
 $C_{out} = AB$

Full Adder



C _{in}	A	B	C _{out}	S
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

$S = A \oplus B \oplus C_{in}$
 $C_{out} = AB + AC_{in} + BC_{in}$

3

FIXED STUDENT TURNS

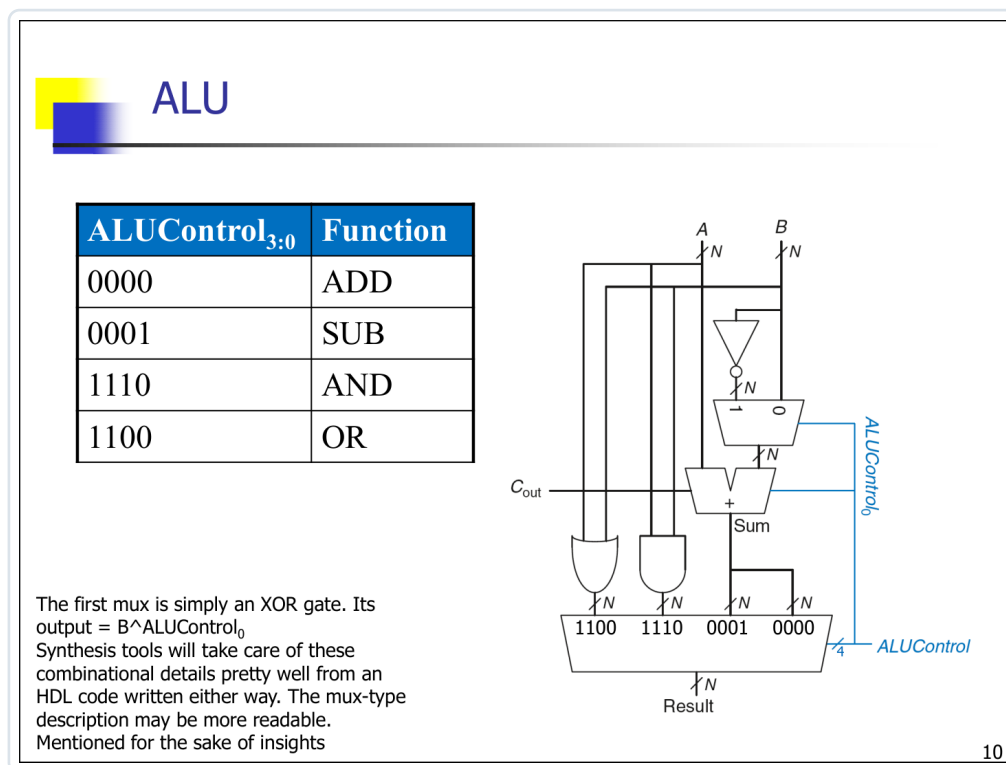
1. What do A, B, Cin, S and Cout represent on these two symbols? I'm trying to understand what each one actually does.
2. For A = 1 and B = 1, I expected S to be 2. Where does that 2 go in the outputs?
3. If Cin is also 1 on the full adder, which outputs change? Could the half adder handle that same input?
4. If I had to add two two-bit numbers, would the carry from the first column go into the next half adder, or would that next column need a full adder? Why?
5. I think the half-adder carry is A OR B, but for the full adder I wrote Cout = AB + ACin + BCin. Could you check those against input cases and show the resulting truth-table rows?

EXPECTED ANSWERS - researcher only; hide from tested models

1. A and B are the one-bit operands. S is the sum bit and Cout is the carry into the next column. The full adder also has Cin, an incoming carry; the half adder does not.
2. 1+1=2, or binary 10: the low bit appears as S=0, and the high bit appears as Cout=1. A one-bit S port cannot hold the number 2 by itself.
3. With A=B=Cin=1, the total is binary 11, so S=1 and Cout=1. The half adder cannot accept a separate Cin; it only represents the Cin=0 case for the same A and B.
4. For two bits with no external carry, the least-significant column can use a half adder. Its carry must enter the next column, which needs a full adder to combine its A, B and Cin inputs.
5. The half-adder carry is A AND B, not OR: A=1, B=0 disproves OR. The proposed full-adder carry AB+ACin+BCin is correct; S=A XOR B XOR Cin. In A,B,Cin → S,Cout order the full-adder rows are 000→00, 001→10, 010→10, 011→01, 100→10, 101→01, 110→01, 111→11; the half-adder A,B → S,Cout rows are 00→00, 01→10, 10→10, 11→01.

Why does one ALU control bit go to two places?

Week 6/Chapter 4 Arithmetic for Computers.pdf, p.10. Keep the ALU circuit and control table; crop the paragraph explaining the first mux. Make ALUControl[3:0], the bit-0 fan-out, the four-way output mux, and the Cout port readable.



FIXED STUDENT TURNS

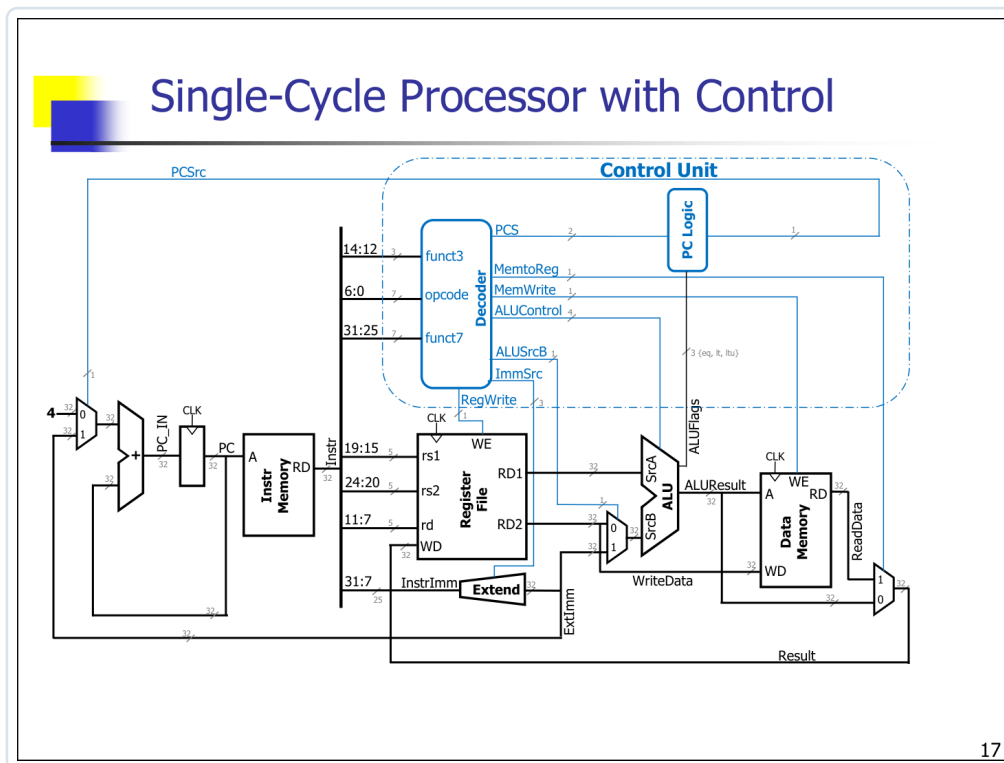
1. I see four operation codes in the table, but ALUControl[3:0] looks like four bits. Am I counting the bits correctly?
2. Which end is bit 0 here, and where does that bit go in the drawing?
3. That little block before the adder seems to choose B or inverted B. Why does the same bit also feed the carry-in of the adder?
4. If $A = 7$ and $B = 3$, follow the subtraction code through both of those bit-0 connections. What result should reach the output mux, and what would go wrong if we inverted B but left carry-in at 0?
5. Since the table shows only four operations, could the whole control be two bits instead? Or would that require changing the wiring and the codes drawn here?

EXPECTED ANSWERS - researcher only; hide from tested models

1. Yes. [3:0] names bits 3, 2, 1 and 0, a four-bit signal. The table shows four *used codes* out of 16 possible four-bit patterns; four operations do not imply that this circuit has only two control wires.
2. Bit 0 is the rightmost bit. The pictured ALUControl₀ line fans out to the B-versus-inverted-B choice before the adder and to the adder carry-in; the full control also selects the output-mux entry.
3. For SUB (0001), bit 0 selects $\sim B$ and supplies carry-in 1, implementing two's-complement subtraction $A + (\sim B) + 1$. ADD (0000) selects B and carry-in 0.
4. $7 - 3 = 4$, so SUB selects the adder's sum 4 at the output mux. Inverting B but adding 0 instead of 1 yields $7 - 3 - 1 = 3$ in the fixed-width arithmetic, an off-by-one result.
5. A *redesigned* four-operation interface could encode the choices in two bits, with new decoding to drive the complement, carry-in and output mux. Merely shortening the control word on this drawing would break the listed codes and bit-0 wiring.

Where does the immediate go in `addi`?

Week 4/Chapter 3B RISC-V Microarchitecture.pdf, p.17. Keep InstrImm, Extend, ExtImm, both ALU inputs, ALUSrcB, WriteData, and the result mux. Remove the control table from p.18.



FIXED STUDENT TURNS

1. For `addi x5, x5, 5`, if `x5` starts at 7, what is the difference between `InstrImm` and `ExtImm`, and where does the 5 go after `Extend`?
2. Does `ALUControl` choose the two ALU inputs? I called `ALUSrcB` a control register in my notes; is that the right term for the line going to the mux?
3. I thought `ExtImm` was for instructions like `addi`. Why does it also connect to the adder near the PC? Does `addi` use that route?
4. Now change it to `add x5, x5, x6`. Which ALU input path switches, and what has to be selected for writeback in each case?
5. `RD2` also runs toward `WriteData` at data memory. For this `addi`, does that mean it stores anything there, or is that wire just available for a different instruction?

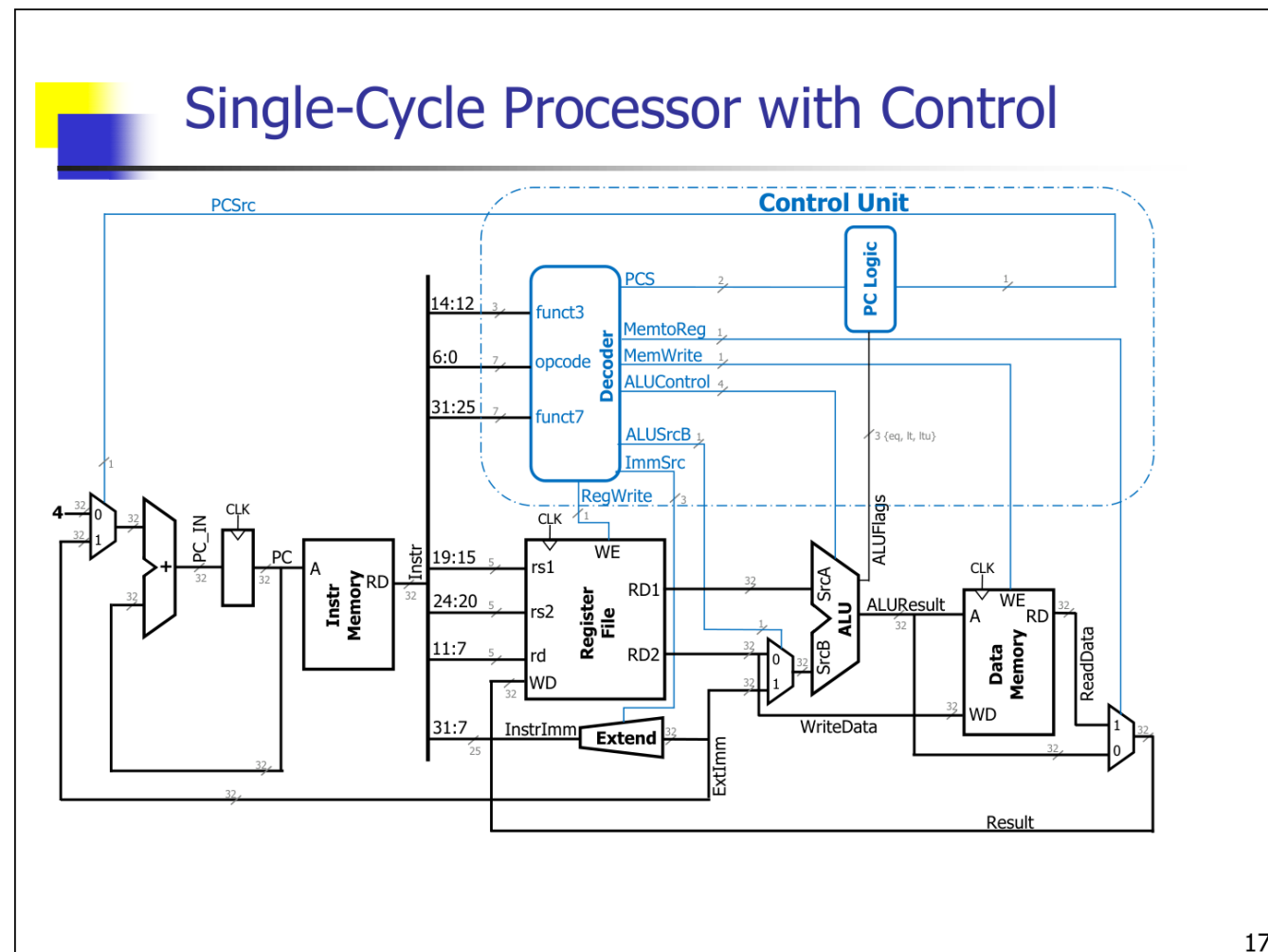
EXPECTED ANSWERS - researcher only; hide from tested models

1. `InstrImm` denotes bits taken from the instruction; `Extend` interprets them according to `ImmSrc` and produces 32-bit `ExtImm`. The ALU B mux selects that immediate, `RD1` supplies `x5=7`, and the ALU/writeback route produces `x5=12`.
2. `ALUControl` chooses the ALU operation, here addition; `ALUSrcB` chooses `RD2` versus `ExtImm` as the second operand, here `ExtImm` when the signal is 1. It is a control signal, not a storage register.
3. `ExtImm` is also wired to the PC-relative target adder for instructions such as branches and `jal`. For ordinary `addi`, the PC selection stays on `PC+4`; its immediate goes through the ALU B mux, not into the chosen next-PC path.
4. `add x5,x5,x6` selects `RD2` (`x6`) with `ALUSrcB=0`; `addi` selects `ExtImm` with `ALUSrcB=1`. Both select the ALU result for register writeback and assert `RegWrite`; `x6`'s value was not specified, so an exact sum for `add` cannot be given.
5. The `RD2`-to-`WriteData` wire is available to sw, but `addi` has `MemWrite=0`, so data memory is not changed. `addi` selects the ALU result, not data-memory `ReadData`, for writeback.

Can these two processors run the same instructions?

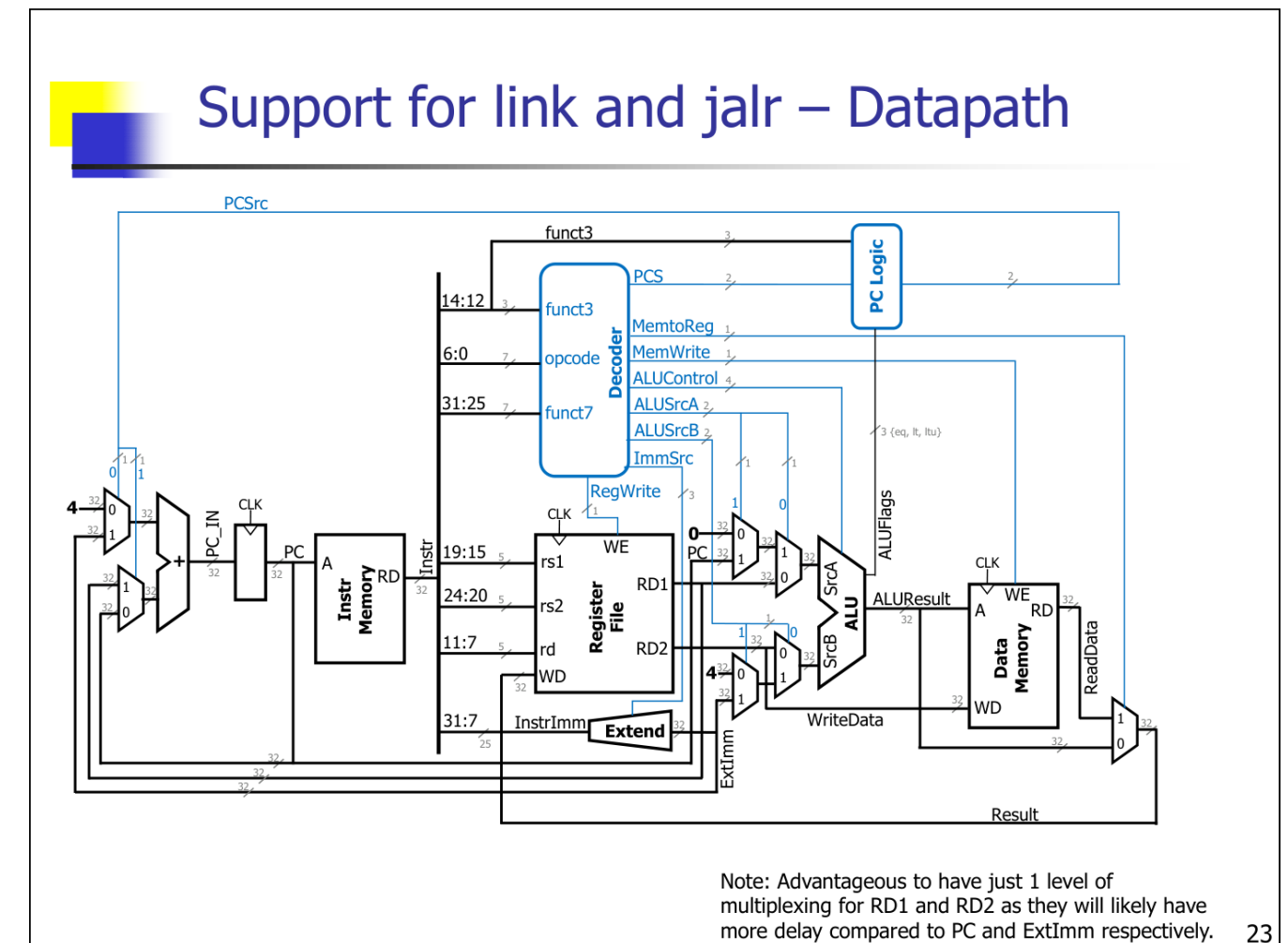
Week 4/Chapter 3B RISC-V Microarchitecture.pdf, p.17 (earlier processor with control), and p.23 (expanded processor with link and jalr). Crop the slide headings and bottom note, retain all mux inputs, control widths, PC adder, register writeback, and memory ports. Re-render each at sufficient size; a crowded side-by-side thumbnail is inadequate. Supply an instruction list on the question page, not an answer table.

A - earlier, p.17



17

B - expanded, p.23



23

FIXED STUDENT TURNS

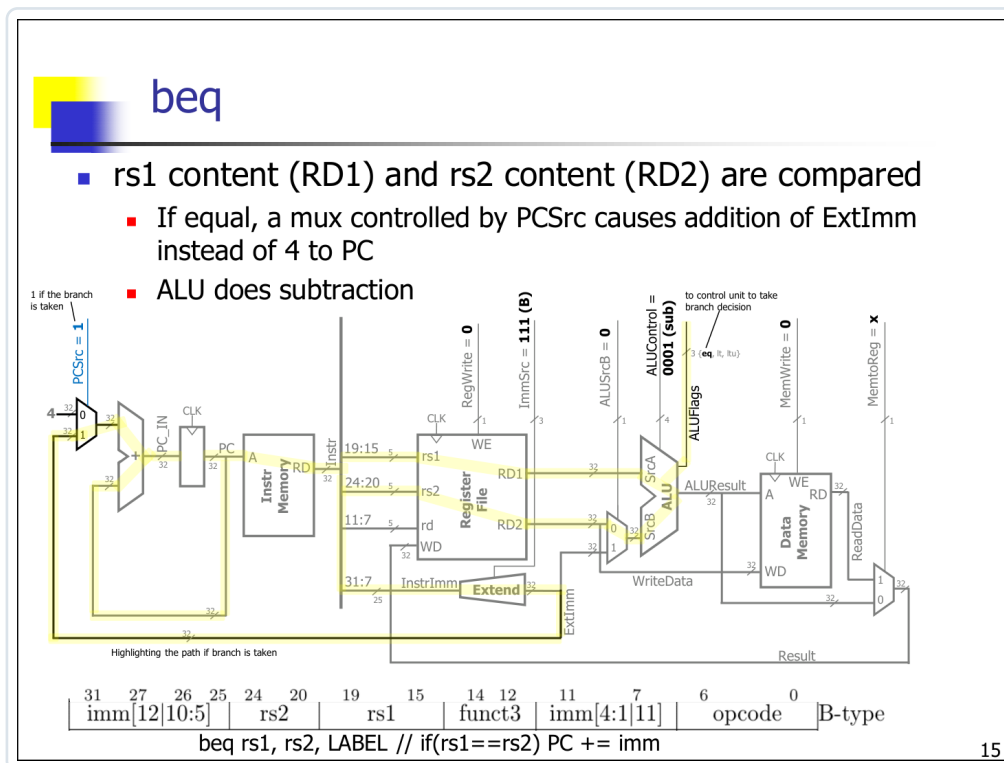
1. Can both A and B run addi x5, x6, 5? What in each picture tells you?
2. What about lw x5, 8(x6)? Both have data memory, but which value goes to its address input and which value eventually gets written into x5?
3. What about lui x5, 0x12345? Can both do that too, or is something missing from one of them?
4. I can see a PC-plus-offset route in A. Does that mean A can do jal x1, 12 completely, including saving the return address? Compare it with jal x0, 12 and point to the path B adds.
5. Now try jalr x1, 8(x2). Which diagram can make the next PC depend on x2 and save PC+4 into x1? Trace both outputs, and tell me what is missing in A.

EXPECTED ANSWERS - researcher only; hide from tested models

1. Yes. In A and B, RD1 reads x6, Extend supplies 5, ALUSrcB selects the immediate and the ALU adds them. The writeback selection chooses ALUResult and RegWrite writes x5; B's extra input muxes can select the same ordinary path.
2. Yes. RD1 gives x6's base value, the extended 8 is added by the ALU, and ALUResult addresses data memory. The memory's ReadData, not the address or the literal 8, is selected for writeback to x5 in both; no memory contents were specified, so no numeric x5 result is known.
3. Only B supports the pictured lui route: its ALUSrcA can choose zero, its B input can take the extended U-immediate, and addition passes 0x12345000 to x5. A's ALU A is fixed to RD1 and has no zero choice; merely changing the decoder cannot create that data path.
4. A has PC+offset selection, so jal x0,12 can perform the jump without needing to retain a link. It cannot implement jal x1,12 fully because PC+4 cannot be selected into register writeback; B adds that link path. The next PC also requires the decoded J offset, as assumed in these figures.
5. B selects RD1 from x2 as the target-adder base, adds the extended 8, selects the result for the next PC, and separately writes PC+4 into x1. A's target adder starts at PC, not RD1, and A has no PC+4 register-writeback choice. No numeric next PC or link is possible without x2 and current PC values.

Trace both outcomes of `beg`

Week 4/Chapter 3B RISC-V Microarchitecture.pdf, p.15. Redraw the comparator/ALU flag, PC-plus-4 route, PC-plus-branch-offset route, and PC selection without highlighted answer paths. State that the branch immediate has already been decoded to a byte offset.



FIXED STUDENT TURNS

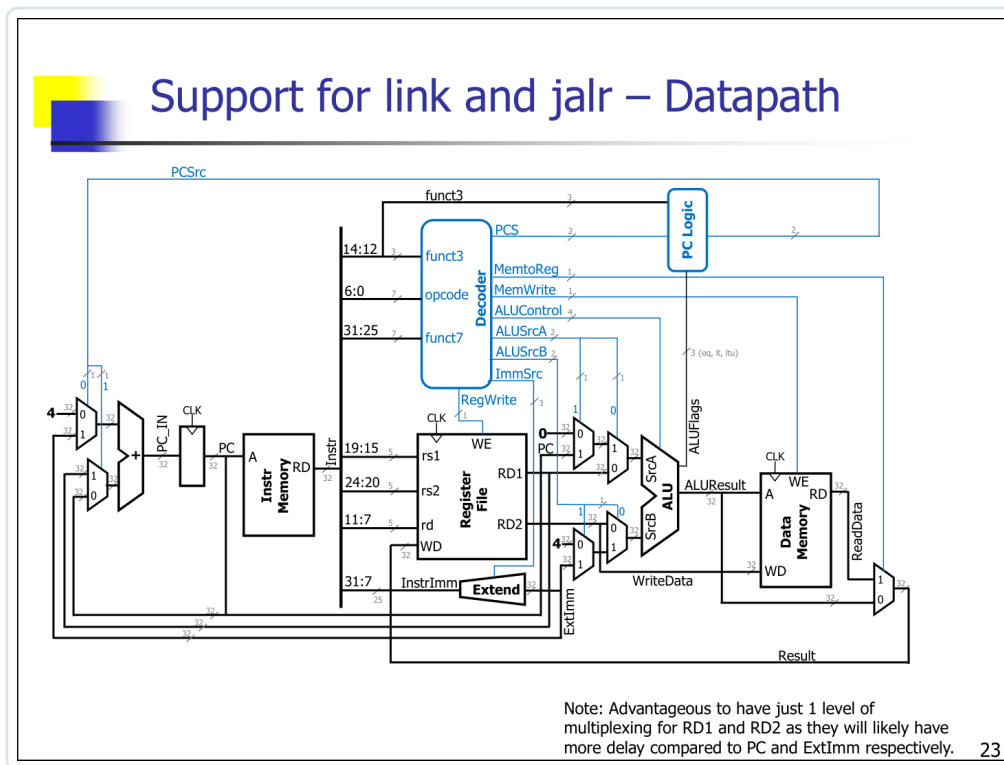
1. For beq x1, x2, 12, where do the values from x1 and x2 enter the comparison, and what output tells the PC logic whether they match?
2. Say the PC is 0x100 and the values match. Which input of the PC mux is selected, and what is the next PC?
3. Keep the same PC and offset but make x2 different from x1. Which path is selected now, and what is the next PC?
4. Does the ALU have to literally subtract to test equality here, or can we tell only that its equality flag is used? How would we avoid reading more into the drawing than it shows?
5. After either outcome, does the ALU result or the branch offset get written to a register? Which control path settles that?

EXPECTED ANSWERS - researcher only; hide from tested models

1. The register file presents x1 on RD1 and x2 on RD2 to the ALU. Its eq flag goes to PC Logic, which controls the next-PC selection for beq.
2. Equality makes the taken target win. With a *decoded 12-byte offset*, PC+12 is 0x10C (hex 0x100+0xC), selected instead of PC+4.
3. If values differ, eq=0; the PC mux keeps the sequential PC+4 path. From 0x100, the next PC is 0x104.
4. This visible datapath guarantees the equality flag is used, but does not expose the ALU's internal equality implementation. The accompanying control table on p.18 specifies SUB for the branch, so subtraction is warranted only when that control is part of the evidence.
5. Neither result is written to a general register for beq: RegWrite=0. The offset helps form the target PC; the comparison flag controls selection. MemWrite=0 also prevents a memory store.

Two values leave a `jalr`

Week 4/Chapter 3B RISC-V Microarchitecture.pdf, p.23. Keep the PC target choices and the PC+4-to-register writeback path; crop the slide's bottom note.



FIXED STUDENT TURNS

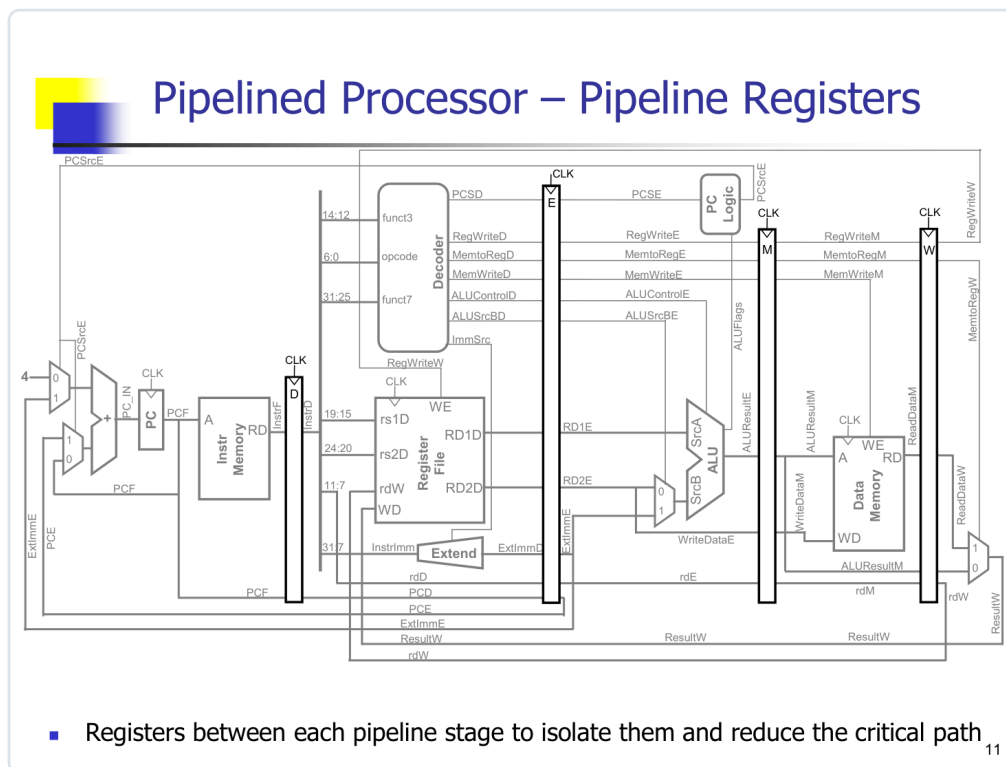
1. For jalr x1, 8(x2), if PC is 0x40 and x2 contains 0x100, what becomes the next PC and what goes into x1? Could you trace each route?
2. The target uses x2, but x1 receives PC+4. Which two mux choices make those different answers possible?
3. If x2 changes to 0x200, which output changes? Would the value written into x1 change too?
4. What if the offset were 9 instead of 8? Is 0x109 the actual next PC under RISC-V, and is the low-bit rule drawn explicitly here?
5. If this were jal x1, 8 instead, which value would be the target base, and would the link value still be PC+4?

EXPECTED ANSWERS - researcher only; hide from tested models

1. The target base comes from RD1=x2=0x100; adding 8 yields next PC 0x108. Separately PC+4=0x44 is selected for register writeback into x1. The target is even, so the ISA's bit-zero clearing does not alter it.
2. The target-base choice selects RD1 rather than PC for the PC adder, while the register-result choice selects PC+4 rather than ALU/memory data. These are distinct routes to PC_IN and the register file's WD.
3. The target becomes 0x208 when x2=0x200. The link remains 0x44, because current PC is still 0x40 and its PC+4 route does not use x2.
4. Under the RISC-V ISA, jalr clears bit 0 of (rs1+imm), so 0x109 becomes 0x108. The teaching datapath does not clearly draw the bit-clearing logic, so a good explanation separates ISA behaviour from the schematic's simplified path.
5. jal uses current PC, not x2/RD1, as target base; with the stated offset the target would be 0x48. It still writes PC+4=0x44 to x1 in B.

Which values do pipeline registers hold?

Week 8/Chapter 5 The Processor (Pipelined RV).pdf, p.11. Keep readable stage boundaries, register banks, instruction and data routes.



FIXED STUDENT TURNS

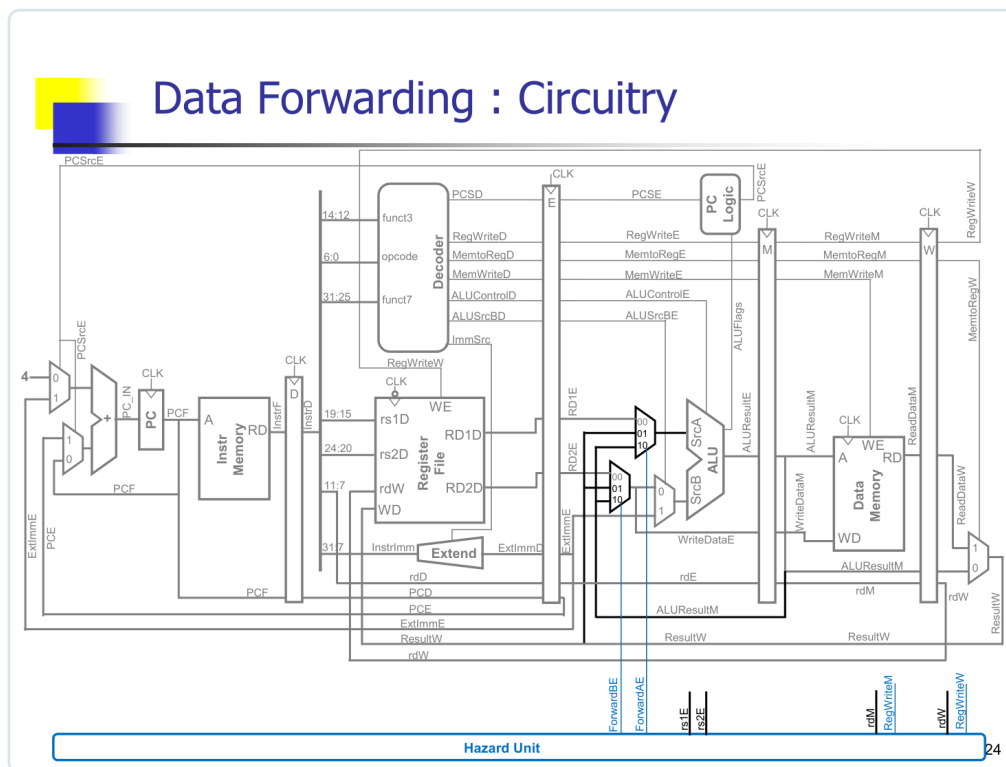
1. I can see four narrow rectangles between the main parts of this processor. What do those rectangles do?
2. Okay, for add x5, x1, x2, what would the rectangle just after the register file need to hold so the instruction can carry on?
3. For lw, is the ALU's address already the value that Writeback needs? Follow where the loaded data actually appears.
4. If I remove the register between Execute and Memory, which work is now combined into one stage, and what happens to the clean clock boundary?
5. So does the first lw necessarily finish in fewer cycles after pipelining, or is the benefit that different instructions can occupy different stages together?

EXPECTED ANSWERS - researcher only; hide from tested models

1. They are pipeline registers (the D, E, M and W boundaries). At clock edges they hold the data and control signals an instruction needs in the next stage, so several instructions can occupy different stages without their values being mixed up.
2. The rectangle after Decode must retain the values read from x1 and x2 (RD1 and RD2), destination ID x5, and the controls needed later, including ALU operation and register write. The RD1E, RD2E, rdE and E-suffixed control labels show these values on the Execute side.
3. No. The lw ALU computes an address in Execute; data memory returns ReadDataM in Memory. That value crosses into W and the result mux selects memory data for register writeback, not the ALU address.
4. Removing the E/M register eliminates that clock boundary: ALU execution and data-memory work would be in one longer combinational stage (and their controls/data must still be carried coherently). The drawing alone supplies no exact new clock period.
5. The first load does not necessarily finish in fewer cycles; in this five-stage picture it passes through multiple stages. The potential gain is throughput from overlapping instructions, subject to stalls and clock timing.

Where does a forwarded value enter?

Week 8/Chapter 5 The Processor (Pipelined RV).pdf, p.24. Keep the M and W bypass lines, ALU input muxes, hazard logic, and stage labels.



FIXED STUDENT TURNS

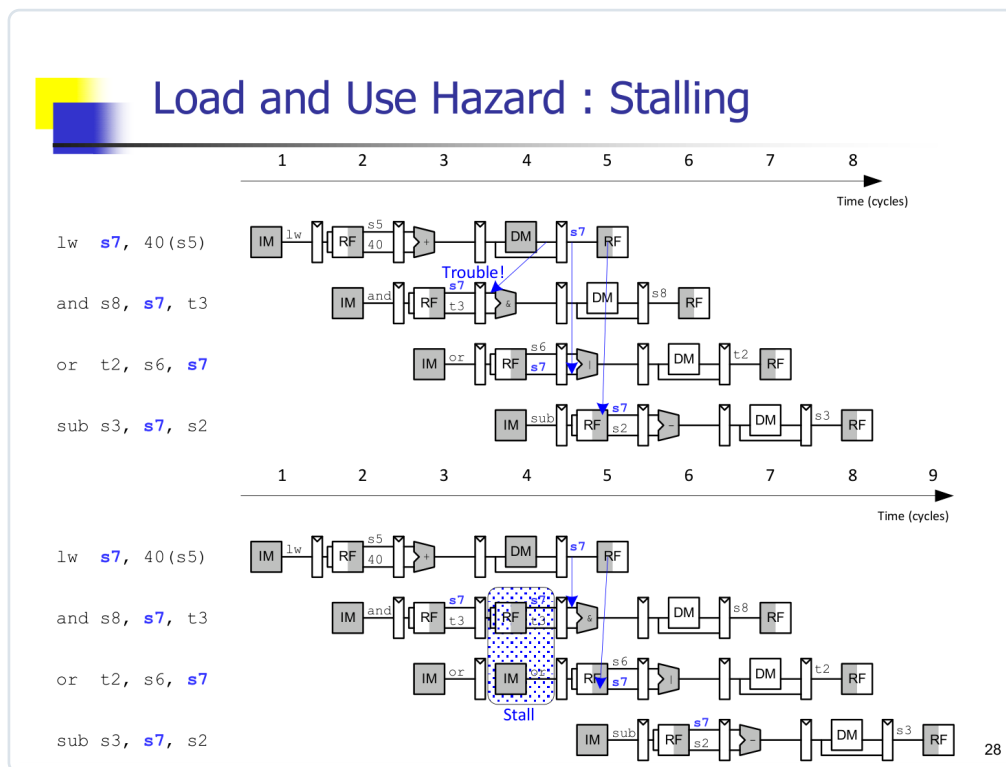
1. If sub x8, x7, x3 comes right after add x7, x1, x2, won't sub read the old x7? How does this picture deal with that?
2. Which ALU input mux needs to change for that sub, and why doesn't the whole instruction have to start over?
3. Suppose two older instructions both write x7. If their results are present in M and W, which version should the consumer use? What in the forwarding controls makes that priority possible?
4. Replace the producer with lw x7, 0(x1). Is the value at the M-stage ALU output now the loaded word or just its address?
5. Can the very next dependent instruction always run without a bubble in that load case? Use the positions of data memory and the ALU input to explain.

EXPECTED ANSWERS - researcher only; hide from tested models

1. The producer's ALU result can bypass the register file from a later pipeline stage to the consumer's Execute ALU input. sub uses the forwarded x7 in place of the stale RD1 value read in Decode.
2. Because x7 is sub's first source, the SrcA/forward-A mux at Execute selects the bypassed value. Decode's old read is overridden before the dependent ALU operation; the instruction has not already executed and need not restart.
3. Choose the younger matching producer in M over an older one in W, provided the M value is a valid result for forwarding. The hazard/forwarding comparisons and mux controls must prioritize that match; otherwise W would feed stale x7.
4. For lw, the ALU output in M is the effective $*address* \times 1 + 0$. The loaded word emerges from data memory later, so forwarding that ALU output as x7 would substitute an address for the data.
5. No. In this five-stage timing, an immediately following consumer wants its operand in Execute before the load's memory read has made the word available. Hold the consumer for a cycle, insert a bubble, then forward the loaded result when available.

Where does the load-use bubble go?

Week 8/Chapter 5 The Processor (Pipelined RV).pdf, p.28. Use the lower pipeline timeline, hiding the blue answer annotations but keeping instruction/stage names.



FIXED STUDENT TURNS

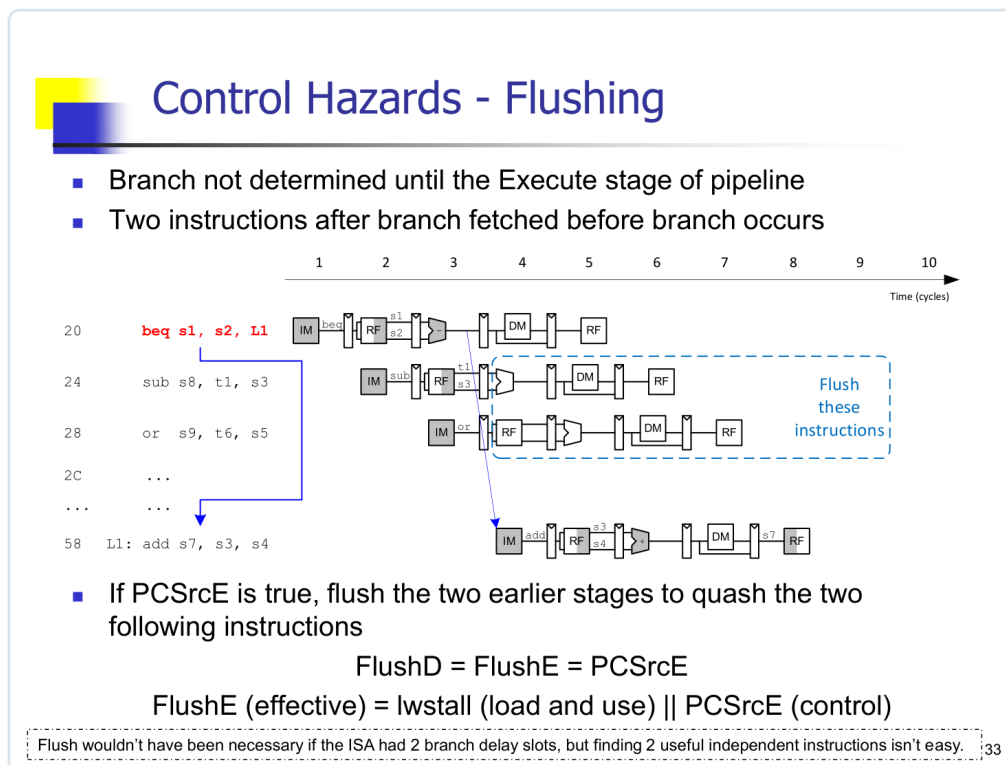
1. Why can't the `and` just continue right after the `lw` in this timeline? They seem to be using different parts of the processor.
2. Could we forward the value at the load's ALU output like we did for an ordinary `add`, or what does that ALU output represent here?
3. What is held back, and where is the empty slot inserted in the lower timeline? Does the load itself stop?
4. There is also `or t2, s6, s7` later. Does its dependence on `s7` force *another* bubble, or can the spacing in this actual timeline already solve it?
5. If the `and` did not use `s7` at all, which part of this depicted stall would no longer be needed?

EXPECTED ANSWERS - researcher only; hide from tested models

1. `and` wants `s7` at its Execute ALU input in the cycle immediately following the load's Execute stage. The load only obtains the word at the end of its Memory stage, too late for that same-cycle input without the depicted stall.
2. The load ALU computes address `s5+40`; it is not the memory word stored at that address. An ALU-result bypass would forward the address, unlike an `add` producer whose ALU output is already the answer.
3. The dependent `and` is held in Decode (and the front of the pipeline is held), while a bubble enters Execute. The older load continues through Memory and Writeback; the phrase 'stall the load until `AND` finishes' reverses the roles.
4. No automatic second bubble follows merely because `or` also uses `s7`. In the lower timeline its later Execute position allows the loaded value to be available, with forwarding or register writeback as depicted; judge the actual slots rather than the register name alone.
5. Without an `s7` dependence, `and` could advance into Execute on the next cycle; the specific load-use hold and bubble would be unnecessary. Other hazards, if present, would be separate questions.

Which instructions get flushed?

Week 8/Chapter 5 The Processor (Pipelined RV).pdf, p.33. Retain the timeline and branch-resolution stage; remove the “Flush these instructions” text and answer arrow.



FIXED STUDENT TURNS

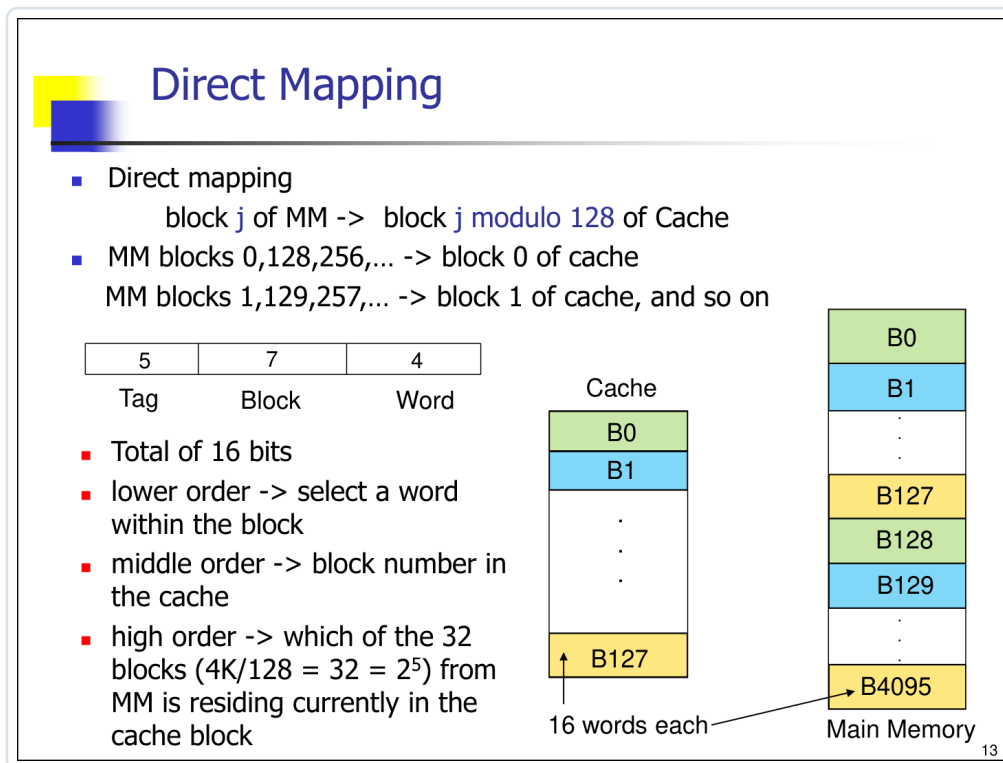
- At the moment this beq is resolved as taken, which younger instructions are already in the pipeline? Point them out on the timeline.
- They have been fetched, so what would go wrong if we let them reach Writeback before changing the PC?
- Does the flush remove beq itself, the two younger instructions, or everything that was fetched earlier?
- If the same comparison comes out not taken, which of those instruction slots remain valid in this scheme?
- In the load-use timeline we inserted a bubble. What changes in this branch timeline instead, and why?

EXPECTED ANSWERS - researcher only; hide from tested models

- The two sequential, younger instructions following beq occupy the stages behind it when its outcome is known in Execute; identify those specific instruction rows/slots in the shown timeline, rather than older rows ahead of the branch.
- A taken branch changes the required instruction address. Those younger sequential instructions came from the wrong path; allowing either to change a register or memory would commit work the program did not choose.
- Squash the two younger wrong-path instructions (invalidate their pipeline entries). The resolving beq and older instructions remain valid and continue; a flush is not a reset of the whole pipeline.
- If not taken, the already fetched sequential instructions are on the correct path and normally remain valid in this pictured scheme. There is no taken-branch flush for them.
- The load-use stall holds a valid dependent instruction until its data can arrive; the taken-branch flush discards younger instructions whose PC path is wrong. Both may create empty slots, but their cause and the affected instructions differ.

Why do two memory blocks fight for one line?

Week 11/Chapter 7 Memory System Principles.pdf, p.13. Remove worked mappings. State 4096 memory blocks, 128 direct-mapped cache lines, 16 words per block.



FIXED STUDENT TURNS

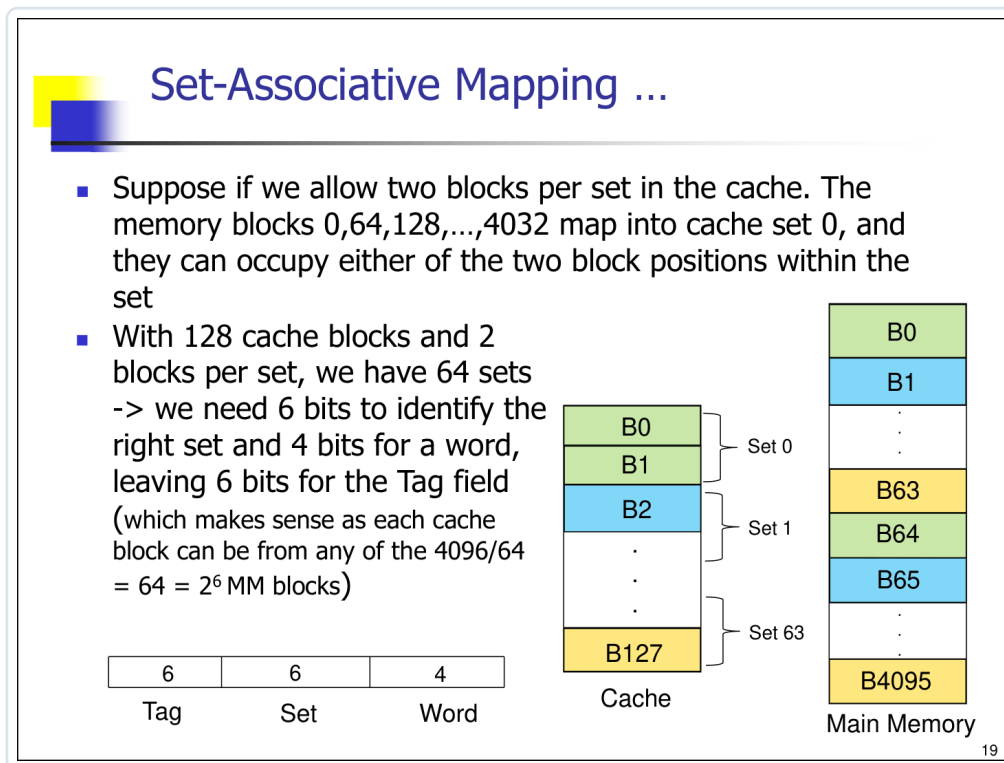
1. If I request memory block 129, where could it be placed in this cache?
2. If block 1 is already there, can blocks 1 and 129 stay in cache at the same time? Which address bits would distinguish them?
3. Now access blocks 1, 129, 1, 129. What happens at that line each time, assuming the cache starts empty?
4. Would choosing a different word *within* block 129 send it to another line, or only change the word offset?
5. If this cache had twice as many lines but the same block size, would 1 and 129 still collide? Which field changes width?

EXPECTED ANSWERS - researcher only; hide from tested models

1. In a 128-line direct-mapped cache, line is block number modulo 128: $129 \bmod 128 = 1$. Block 129 has only line 1 as its slot.
2. They cannot coexist in this one-way line: block 1 also maps to line 1. The tag distinguishes the different memory blocks that share index 1; with 4096 memory blocks, the block address has 12 bits, split into 5 tag bits and 7 index bits.
3. Starting empty, each of the four accesses misses: load 1, replace with 129, replace with 1, replace with 129. Each replaces the line-1 contents, assuming no other cache mechanism or interference changes the trace.
4. Changing the word within block 129 changes only its 4-bit word offset among 16 words. Its memory-block ID, line index 1 and tag remain the same.
5. With 256 lines, block 1 maps to line 1 and 129 to line 129, so they do not collide. The index widens from 7 to 8 bits and the 12-bit block tag narrows from 5 to 4 bits, keeping the 16-word offset at 4 bits.

What does two-way placement change?

Week 11/Chapter 7 Memory System Principles.pdf, p.19. Remove worked mapping text. State 128 cache blocks arranged in 64 two-way sets; 16 words per block; 4096 memory blocks.



FIXED STUDENT TURNS

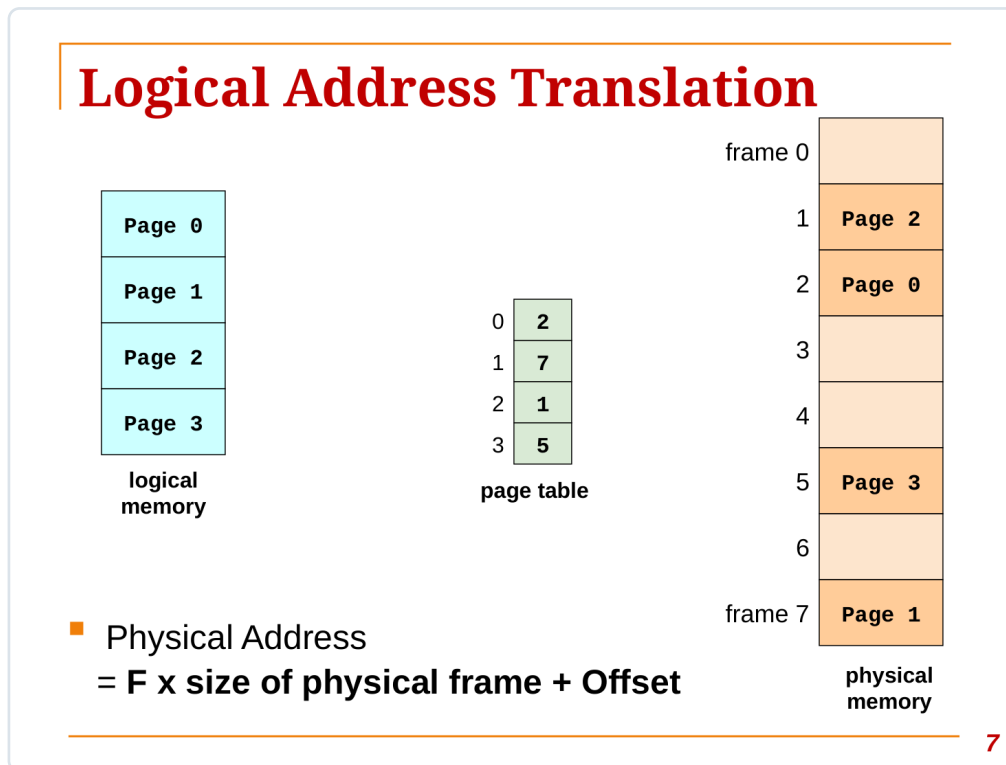
- Where can block 128 go here? Show the set number and the two physical slots available inside that set.
- If block 64 is already in one way of that set, could block 128 occupy the other without evicting it?
- Why does the set field now need six bits instead of the seven line bits in the direct-mapped cache?
- Now request block 0 as well. Can 0, 64 and 128 all remain there? If not, can this drawing alone tell us the exact victim?
- Compare the 1,129,1,129 access pattern from the previous diagram. Does this two-way organisation still force them to evict one another?

EXPECTED ANSWERS - researcher only; hide from tested models

- $128 \bmod 64 = 0$, so block 128 maps to set 0 and may occupy either of its two ways; it is not free to go to an arbitrary set.
- Yes. Block 64 also maps to set 0, but with just that one way occupied, block 128 can fill the other way without eviction.
- There are 64 sets, so set selection needs $\log_2 64 = 6$ bits. The earlier 128 direct-mapped lines needed 7 index bits; the extra physical choice here is a way within the selected set, not another set bit.
- 0, 64 and 128 all map to set 0, which holds only two blocks, so one must be absent or evicted when the third is installed. The exact victim is unknown without current contents and a replacement policy.
- Blocks 1 and 129 both map to set 1 but can coexist in its two ways. With only those two alternating and no interfering access, the first two accesses miss and subsequent ones hit, assuming the cache starts empty.

Which physical frame contains the page?

Week 10/CG227Lect8.pptx, slide 7, "Logical Address Translation". Keep page table 2,7,1,5 and frames; remove the printed formula. Give page/frame size = 256 bytes. For turn 4, a clean second image with page 2's entry changed to 4 is preferable to a text-only edit.



FIXED STUDENT TURNS

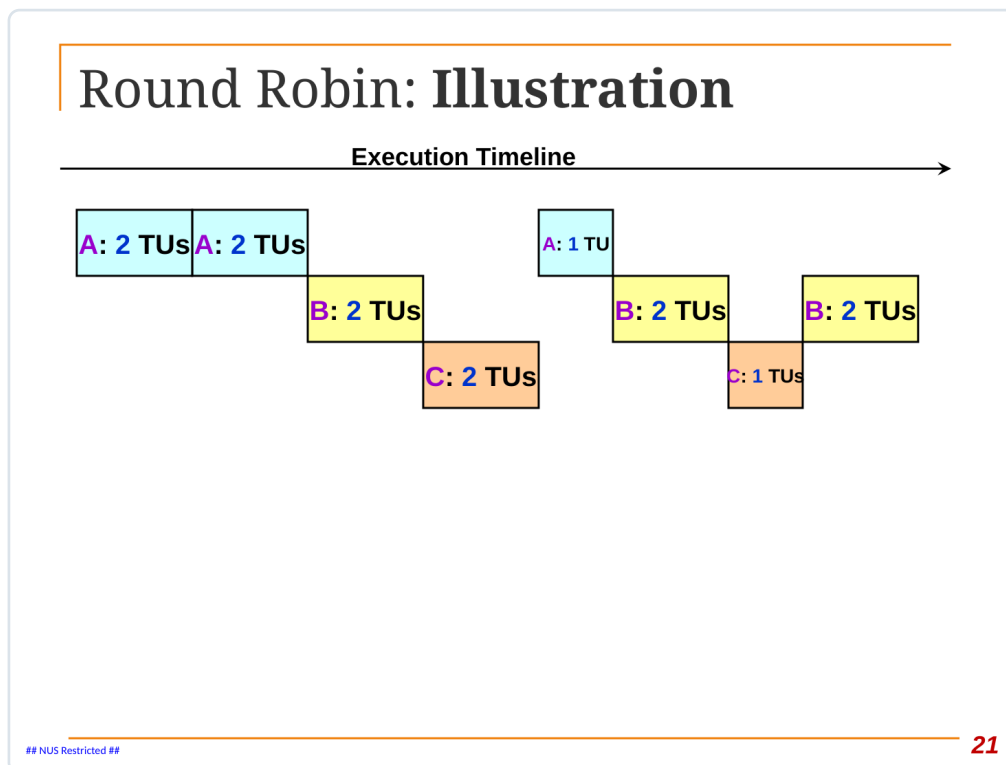
1. If the CPU asks for logical page 2, offset 10, where does that land in the physical memory pictured?
2. I got $2 \times 256 + 10$. Which number in the diagram should replace that first 2, and why?
3. If I choose offset 200 on the same logical page, does the physical frame change, or just the place within it?
4. Now the page-2 entry is changed from frame 1 to frame 4. Which part of the physical address changes, and what does it become for offset 10?
5. If the entry were marked not present instead, could we still take the same arrow straight to a frame?

EXPECTED ANSWERS - researcher only; hide from tested models

1. Use logical page 2 to index the table; its entry is physical frame 1. At 256 bytes per frame, physical address is $1 \times 256 + 10 = 266$.
2. The first number is the *frame* number from page 2's table entry, namely 1; the logical page number 2 chooses the entry but is not itself the physical frame.
3. Frame 1 remains selected. Offset 200 selects another byte within that frame, at $1 \times 256 + 200 = 456$.
4. Changing the entry to frame 4 changes only the frame-number part; offset 10 stays 10. The new physical address is $4 \times 256 + 10 = 1034$ if the modified page is present.
5. No. A not-present entry provides no current resident frame to combine with the offset; address translation triggers handling such as a page fault before an access can use a physical frame.

What can we infer from this round-robin timeline?

Week 4/CG2271Lect3.pptx, slide 21, "Round Robin: Illustration". Preserve execution blocks and durations. Do not supply arrival times the source lacks.



FIXED STUDENT TURNS

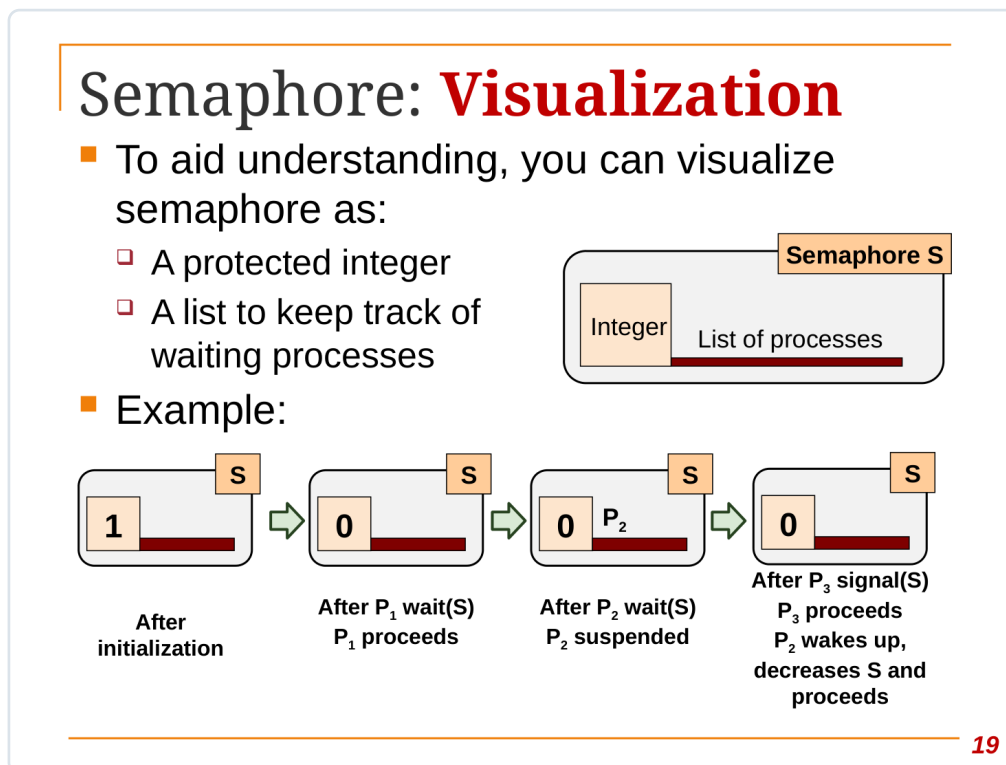
1. A appears in two consecutive 2-TU slots. Does the picture tell us why B and C did not run between them?
2. One possible explanation is that B and C were not ready yet. Can we establish that from this timeline, or is it just a possible ready-queue history?
3. A's final slot is only 1 TU. How could that happen with a 2-TU quantum?
4. If B had already been waiting before A's second slot, would the shown order still look like ordinary round robin? What assumption would we need to check?
5. What extra labels or another diagram would let us reconstruct the queue and waiting times exactly?

EXPECTED ANSWERS - researcher only; hide from tested models

1. No. The timeline shows A's consecutive CPU intervals, but it does not show who had arrived or the ready-queue contents between them. The reason cannot be read uniquely from the blocks.
2. That is a possible explanation, not an established fact. B and C might not yet be ready, but the picture omits their arrival/ready times and queue transitions.
3. A can complete its remaining work after 1 TU and leave before using a full 2-TU quantum. A quantum is a maximum continuous slice, not a requirement to use all of it.
4. If B was ready and ahead of A when A's first quantum expired, ordinary RR would normally give B the next turn; consecutive A would need another explanation. Arrival order and the ready-queue state at that boundary are the crucial missing facts.
5. Provide each process's arrival/ready time, CPU burst or remaining time, initial queue order, and queue changes at quantum expiry/completion (plus switching assumptions if waiting times must be exact). Then the execution and waiting times can be reconstructed.

Where does the second waiter go?

Week 7/CG2271Lect6.pptx, slide 19, "Semaphore: Visualization". Keep S, P₁, P₂, and sequence arrows; crop surrounding explanation.



FIXED STUDENT TURNS

- S starts at 1. Follow P₁'s wait(S) in the picture: who holds the permit after that point?
- When P₂ then reaches wait(S), where does it wait, and does this drawing imply it keeps running on the CPU while waiting?
- In the last panel P₃ calls signal(S). Does that mean P₂ has completed its critical section already, or only that it may resume?
- Suppose nobody signals after P₂ is suspended. Where does the depicted sequence get stuck?
- If S had started at 2, which part of this two-process sequence would change, and would it still enforce one-at-a-time entry?

EXPECTED ANSWERS - researcher only; hide from tested models

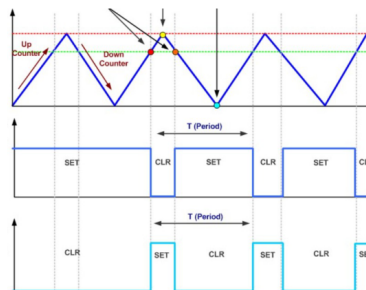
- After P₁'s wait(S) consumes the single available permit, P₁ proceeds and S has no free permit (0 in the pictured convention). The visual doesn't show a separate owner field; 'P₁ holds it' describes the effect of successful acquisition.
- P₂ cannot acquire when the permit is unavailable; the picture places P₂ in the waiting-process list and labels it suspended. In this blocking example it does not repeatedly occupy the CPU to poll S.
- P₃'s signal(S) wakes or enables P₂ to proceed; it does not mean P₂ has already run its critical section, much less completed it. The slide does not explain why P₃ is the process signaling.
- P₂ remains suspended in the waiting list without a signal or another defined way to make a permit available. The diagram cannot show P₂'s later progress from the given events.
- With initial count 2, P₁'s wait would leave one permit, so P₂'s subsequent wait could proceed instead of joining the waiting list. Two concurrent entrants would not enforce a one-at-a-time critical section.

Which crossings change the PWM output?

Week 7/LL-5 PWM Programming.pptx, slide 18. Keep counter triangle, CnV crossing points, both output waveforms, and ELS labels. Crop answer prose.

Choosing PWM Polarity

- ELSnB:ELSnA = 0b10
 - High True: Clear pin matching CnV counting up, set pin matching CnV counting down.
- ELSnB:ELSnA = 0b01
 - Low True: Set pin matching CnV counting up, clear pin matching CnV counting down.



CG2271 Lab Lecture 5

18

FIXED STUDENT TURNS

1. In the top blue output trace, what makes the pin go low and then high again? I'm trying to match it to the triangle above.
2. Do the high-true and low-true outputs have different counter periods in this drawing, or do they mostly invert one another?
3. If CnV moves toward the top of the triangle but the period stays fixed, how do the two crossing times move and what happens to high-true duty cycle?
4. What if CnV is near the bottom instead? Use the two sloped counter segments to check the direction of the change.
5. Can we calculate an exact percentage of high time from this slide, or do we need numerical CnV and period values?

EXPECTED ANSWERS - researcher only; hide from tested models

1. For the top high-true trace (ELS=0b10), it clears to low at the CnV match while the counter counts up and sets back to high at the down-count match. Match the two transitions to the rising and falling slopes.
2. They share the same up/down counter period and CnV crossings. The low-true (0b01) trace performs opposite set/clear actions and is complementary in the depicted steady pattern, not a new timer frequency.
3. Moving CnV upward delays its up-count crossing and advances its down-count crossing. The low interval between those crossings shrinks, so the top high-true output is high for more of each unchanged period.
4. Moving CnV toward the bottom makes the up-count match earlier and the down-count match later, widening the low interval and shortening the high-true pulse. The complementary low-true high interval widens.
5. No exact percentage is warranted from a schematic without numeric compare value and counter limit/period. With those settings and the defined edge behaviour one can calculate duty cycle; the drawn proportions are illustrative.

Where are UART bits sampled?

Week 11/LL-8 UART Programming.pptx, slides 4-5. Prepare a single legible frame with idle, start, payload, optional parity, stop, and sample points. Remove teaching prose.

Receiver Basics

- Wait for a falling edge (beginning of a Start bit)
 - ❑ Then wait $\frac{1}{2}$ bit time
 - ❑ Do the following for as many data bits in the word
 - Wait 1 bit time
 - Read the data bit and shift it into a *receive buffer* (shift register)
 - ❑ Wait 1 bit time
 - ❑ Read the bit
 - if 1 (Stop bit), then OK
 - if 0, there's a problem!

CG2271 Lab Lecture 8
5

FIXED STUDENT TURNS

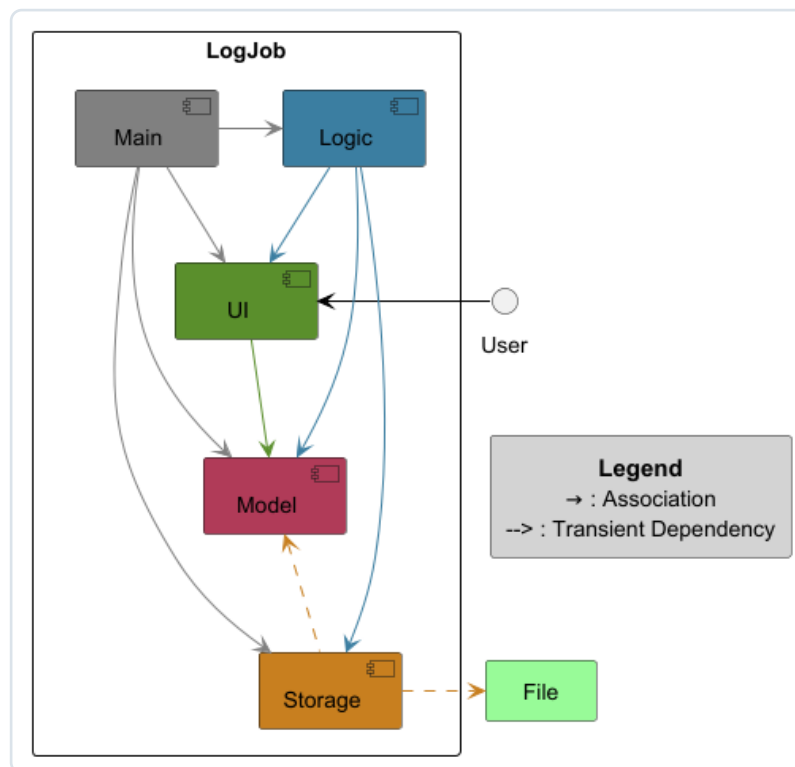
1. Which transition tells the receiver that a frame may have started, and where does it check the start bit?
2. Why are the payload sample points near the middle of each bit rather than right at a transition?
3. If I count the start bit as payload bit 0, how would that shift the rest of the byte compared with the labels in this frame?
4. The drawing includes parity. Is that always part of UART, or does the frame format depend on the configuration?
5. If the receiver's bit clock is a little faster than the transmitter's, where would its later sample points drift relative to the bit centers?

EXPECTED ANSWERS - researcher only; hide from tested models

1. The idle-high line falls to the start-bit low level, signaling a possible frame. The receiver checks near the middle of that start-bit interval to confirm it before scheduling data samples.
2. Transitions are where line voltage and timing uncertainty are greatest. Sampling near each bit's center gives margin against edges and small clock mismatch.
3. The start bit is a framing interval, not data bit 0. Counting it as payload would misalign every later payload sample and reconstruct the wrong bit sequence/byte.
4. Parity is optional and depends on the agreed UART framing configuration; transmitter and receiver must agree whether it exists, as well as data bits and stop bits.
5. A faster receiver uses shorter local bit periods, so its successive sample points drift earlier relative to the transmitter's bit centers; enough drift can sample an edge or neighboring bit and cause data/framing errors.

What does an architecture arrow actually assert?

TP/docs/diagrams/architecture-diagrams/ArchitectureDiagram.png. Keep its legend. For a visual change in turn 4, create an edited second diagram with a direct UI→File arrow.



FIXED STUDENT TURNS

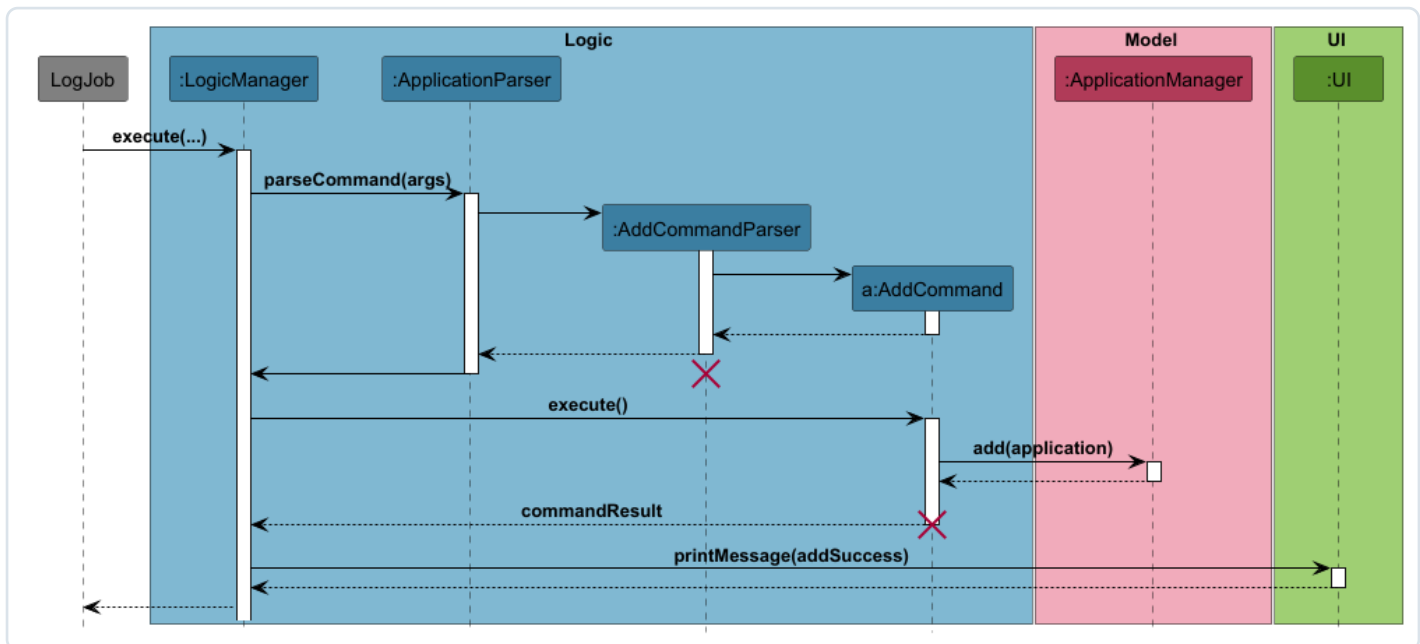
1. The arrow from Logic to Storage goes one way. According to this legend, what relationship does that claim?
2. Does it mean every command must call Storage, or only that Logic can depend on it? What can we safely infer from this drawing?
3. Storage also has a dashed arrow toward File. Does the legend say that line is a different kind of relationship?
4. If I draw a new direct arrow from UI to File, what claim have I added that the original picture does not make?
5. Could this architecture diagram tell me the exact order of calls for an add command, or would a sequence diagram answer that better?

EXPECTED ANSWERS - researcher only; hide from tested models

1. The legend calls the solid Logic→Storage arrow an association: Logic has a directed relationship with Storage. The arrow direction identifies the drawn dependence/connection, not a specific call instance.
2. It does not prove every command calls Storage. It shows the architectural relationship is allowed/present; individual command behaviour requires a runtime trace or code.
3. Yes. The dashed Storage→File arrow is a *transient dependency* according to this drawing's legend, unlike the solid association. The dash does not mean 'unimportant'.
4. A direct UI→File arrow would assert a new direct UI-to-File relationship, bypassing the absence of such an edge in the original. It would not, by itself, prove every UI action accesses a file.
5. No exact order of method calls follows from this static component view. The add sequence diagram can show the ordered message exchange for that use case.

Who actually adds the application?

TP/docs/diagrams/sequence-diagrams/add-sequence.png. Preserve names, message arrows, creation arrows, returns, and activation bars at readable scale.



FIXED STUDENT TURNS

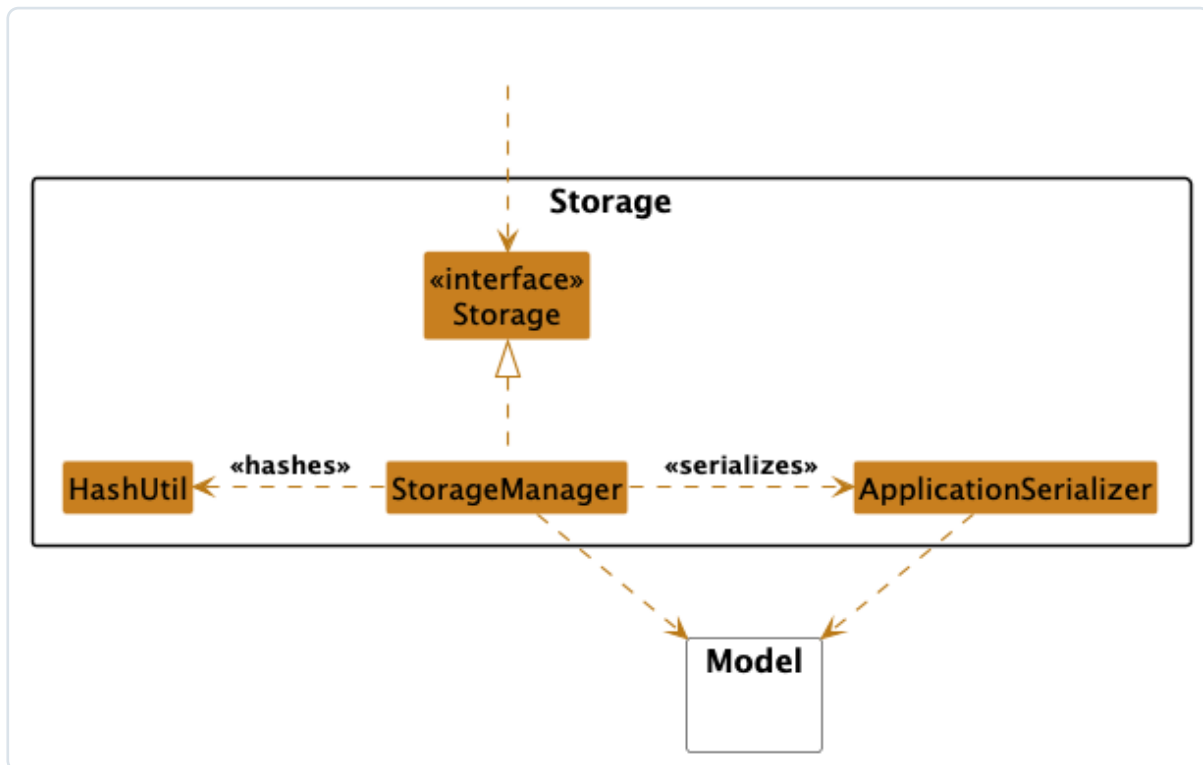
1. When LogJob issues execute(...), who receives it, and which object starts interpreting the command?
2. An AddCommandParser and an AddCommand appear partway down. Which arrows create them, and which object creates which?
3. Follow the actual call that adds an application. Is it made by the parser, the command, or the UI?
4. Trace the path back from that call to printMessage(addSuccess). Does the UI update the Model, or just display the returned result?
5. If ApplicationManager.add(...) were removed from the sequence, would the other arrows alone show that an application was saved? Why?

EXPECTED ANSWERS - researcher only; hide from tested models

1. LogJob sends execute(...) to LogicManager. LogicManager calls ApplicationParser.parseCommand(args) to begin interpreting the command words.
2. ApplicationParser creates the AddCommandParser; the arrow from that parser creates a:AddCommand. They appear after the earlier messages, as the creation arrows show, rather than being assumed present from the top.
3. After parsing returns the command, LogicManager calls execute() on a:AddCommand. That command calls add(application) on ApplicationManager in Model; the parser and UI are not the callers of the actual add.
4. The Model call returns to AddCommand, then a commandResult returns to LogicManager, which calls printMessage(addSuccess) on UI. The shown UI role is display of the success message, not a UI→Model update.
5. No. Other arrows show parsing, command execution and display, but without the ApplicationManager.add(application) message this particular drawing would no longer depict the Model addition. It also does not independently prove persistence to a file.

What can this class diagram prove?

TP/docs/diagrams/class-diagrams/StorageClassDiagram.png. Retain arrowheads, interface marker, dependencies, labels, and Model box.



FIXED STUDENT TURNS

1. What is the hollow triangle between StorageManager and Storage telling me?
2. If another part of the app is written against the Storage interface, which details of StorageManager could it avoid depending on?
3. The arrow labelled <<hashes>> points at HashUtil. Does that show a dependency or ownership? What about the arrows toward Model?
4. If I added a new class implementing Storage, where would its realization arrow point? Would that alone require changing the existing HashUtil dependency?
5. Can this picture establish whether hashing happens before serialization during a save, or is that order outside what a class diagram shows?

EXPECTED ANSWERS - researcher only; hide from tested models

1. StorageManager realizes/implements the Storage interface; the dashed realization line's hollow triangle points toward the interface being implemented, not from interface to implementing class.
2. The caller can depend on the operations promised by Storage rather than the concrete StorageManager type or its HashUtil/serializer implementation details. The interface diagram alone does not list all exact methods.
3. The <<hashes>> arrow from StorageManager to HashUtil and <<serializes>> arrow to ApplicationSerializer indicate use/dependency, not ownership. Both StorageManager and ApplicationSerializer also have directed dependencies toward Model in the picture.
4. A new implementation's realization arrow would point to Storage. That fact alone creates no required HashUtil dependency; whether the new class uses hashing is a separate design decision.
5. No. A class/dependency diagram says what classes relate to or use, not the order of runtime calls. A sequence diagram, code, or another behavioural specification would be needed to say whether hashing precedes serialization.